Platform Infrastructure & Automation - Take-Home Challenge

Senior Platform Engineer @ Arqh

Deliverable: **GitHub Repository + Platform Runbook (PDF/Markdown)**

Expected Timebox: 48–72 Hours

---

1. Context & Objective

Arqh runs a real-time, event-driven logistics dispatch engine utilizing Python microservices, low-latency Redis Streams for processing coordination, and MongoDB as our durable state layer. As our Senior Platform Engineer, your core focus is to design, automate, build, and test a production-grade deployment infrastructure for this topology.

You are provided a baseline application setup modeled directly after our core system. Your job is to construct the entire local Kubernetes architecture, an optimized CI/CD pipeline, an end-to-end integration/smoke testing engine, and a live infrastructure visibility baseline.

---

2. The Core Technical Pillars

Pillar A: Production-Grade Kubernetes Topology

You must establish a local multi-node cluster environment (using `kind` or `minikube`) and author declarative manifests or a unified Helm chart implementing the following patterns:

- **Dynamic Resource Allocation & Scaling:** Configure Horizontal Pod Autoscalers (HPA) targeting standard cluster metrics (CPU/Memory utilization thresholds) for your microservices.
- **Ingress Architecture & Traffic Control:** Set up a production-ready Ingress Controller (e.g., NGINX or Traefik) handling path-based application routing, header re-writes, and basic TLS termination rules.
- **Cluster Resilience:** Define rigid `readinessProbes` and `livenessProbes` tied to external structural backing services (MongoDB/Redis). Enforce explicit container `requests` and `limits` to prevent node exhaustion.

Pillar B: Automated CI/CD Execution Pipeline

Using GitHub Actions, design a completely automated build and packaging pipeline that triggers on all pull requests and merges to your main branch:

- **Build Layer Optimization:** Author multi-stage Dockerfiles that leverage precise layer caching, minimal base images (e.g., alpine or distroless variants), build-time arguments, and secure non-root execution layers.
- **Quality Control Gates:** Integrate automated testing execution passes ensuring code formatting, style verification (linting), and container configuration tests run reliably on every push.

Pillar C: Automated Integration & Smoke Testing

Rather than focusing on application-level logic, you must write automated programmatic infrastructure tests that run post-deployment:

- **Cluster State Validation:** Write an automated test suite (using tools such as Python/`pytest` interacting with the Kubernetes API client, comprehensive Bash frameworks, or Terratest) verifying that pods initialize correctly, secrets are mapped safely, and configurations match expectation.
- **Integration/Smoke Gates:** Programmatically verify end-to-end routing integrity by making external HTTP/WebSocket client requests through the Ingress layer down to the backing data stores, validating response flows.

Pillar D: Observability Primitives & Health Monitoring

Provide complete operational visibility into your running platform:

- **Telemetry & Metrics Sinks:** Provision a localized infrastructure monitoring pipeline (e.g., Prometheus and Grafana or equivalent cloud-native stacks).
- **Operational Health Visualization:** Build a comprehensive dashboard configuration visualizing pod restarts, error logging spikes, cluster capacity limits, and external service dependency response metrics.

---

3. Technical Deliverables

1. **Infrastructure Code Repository:** A clean Git repository housing all manifests, Helm charts, optimized Dockerfiles, and pipeline configuration YAMLs.
2. **The Unified Bootstrap Control Script:** A single automation script (e.g., `./run-platform.sh` or `make bootstrap`) capable of spinning up the local cluster, provisioning dependencies, applying configurations, and ensuring a green state.
3. **The Platform Runbook (`README.md`):** A comprehensive engineering handbook detailing operational setup steps, testing execution syntax, and telemetry access coordinates.

---

4. The Baseline Codebase

To complete this take-home task, you will start with our existing reference codebase repository:

Repository Link: https://github.com/arqh-ai/ArqhPrep

(Note: Please fork this repository into a private repository for your workspace submission. Your submission is to be given access to @erkul-mert and @CanIlgu and below point is ***EXTREMELY*** *important)*

Your Repository Environment Scope

We have prepared this skeleton specifically for infrastructure work by freezing the frontend interface assets and retaining only the operational python/node state services, API routes, and database connectivity.

- **Do Not Build the FULLSTACK:** Focus entirely on packaging, running, and scaling the backend and proxy layers. You do not need to modify user interface code. Y
- **Strip Outdated Setup:** Any baseline local configurations (such as simple development docker-compose files) must be replaced entirely by your production-ready, highly observable Kubernetes topology. You are given an initial starting point with Prometheus, but you should be comfortable stripping that one out for your own (and you are actually encouraged to do so!).

CRITICAL ARCHITECTURAL REQUIREMENT (The Readme Bottom Section)

At the very bottom of your `README.md`, you **must** provide a dedicated architectural comparison, either as a matrix or a table. For **every single non-obvious infrastructure or tooling choice made** throughout this assignment, you should document:

- Why you picked that specific tool, architectural pattern, or configuration strategy.
- A detailed structural comparison against at least one prominent direct alternative option (e.g., Kustomize vs. Helm, NGINX vs. Traefik, custom script vs. Terratest, Prometheus vs. Managed Logs).
- The technical architectural tradeoffs, drawbacks, and operational limitations inherent to your final decision.
- Next steps, and what you think can be improved. Very short, very precise!